Lab 2: Serial connection

Trần Anh Dũng — 20226031
Partner: Maier Benedikt Quirin — 20260015S
Embedded Systems, IT4210E, semester 2026.1
Date: 30 October 2026

Declaration: The content and source code presented in this lab report were completed by me and my partner, Maier Benedikt Quirin. Any content or material referenced from external sources is acknowledged, including the original source and author.

We used the STM32F429 Discovery board, Tiny RTC module, SH1106 OLED and RC522 reader. Exercises 3.6, 3.7 and 3.8 are in the same project, `Lab2_Peripherals`. Exercise 3.9 is in a separate project, `Lab2_AccessControl`.

3.1. Tiny RTC module

Can DS1307 and AT24C32 work at the same time? Why?

Yes. Both chips use I2C and share the SDA and SCL wires. They have different addresses, so the STM32 can choose which chip to communicate with. Only one chip is addressed in a transaction. The DS1307 continues keeping time while the STM32 communicates with another device.

DS1307 is the real-time clock. AT24C32 is an EEPROM with 32 Kbit, or 4,096 bytes, of storage. EEPROM keeps its stored data when power is removed.

Which pins are needed to connect the module?

The power pins are VCC and GND. The communication pins are SDA and SCL. BAT/VBAT is the backup supply for the clock. The DS and SQ pins are not needed to read the date and time through I2C.

For our experiment, VCC was connected to 5V, GND to common ground, SCL to PA8 and SDA to PC9. We removed the coin cell and left BAT unconnected. This means the clock had to be set again after disconnecting USB. Resetting only the STM32 did not remove power from the RTC.

What are the addresses of the two chips?

DS1307 has the 7-bit address `0x68`. On the bus, the address byte is `0xD0` for writing and `0xD1` for reading.

AT24C32 has an address from `0x50` to `0x57`, depending on its A2, A1 and A0 pins. These pins are grounded in the supplied circuit diagram, so its address is `0x50`. The corresponding write and read address bytes are `0xA0` and `0xA1`.

We scanned the actual module and received responses at `0x68` and `0x50`.

Circuit connections and setup

Module pin	STM32 connection
RTC VCC, GND	5V, GND
RTC SCL, SDA	PA8, PC9
OLED VCC, GND	3V, GND
OLED SCL, SDA	PA8, PC9, shared with RTC
RC522 3.3V and RST	Both to 3V
RC522 GND	GND
RC522 SDA/SS	PE4
RC522 SCK	PE2
RC522 MISO	PE5
RC522 MOSI	PE6
RC522 IRQ	Not connected

All modules share ground. We used the matching pins on the RTC module's two headers to share SCL, SDA and GND with the OLED. The RC522 pin labelled SDA is used as SPI chip select here, so it goes to PE4, not PC9.

The circuit was checked with the TA before powering. We left out the external 100 uF capacitor as instructed in the current lab announcement. The RTC battery was removed for these tests.

The CPU clock is 180 MHz. We configured I2C3 on PA8 and PC9 at 100 kHz. The DS1307 datasheet only allows standard-mode I2C up to 100 kHz, even though a screenshot in the handout shows 400 kHz. SPI4 uses mode 0, 8-bit transfers and prescaler 64. PE4 is an output and starts HIGH. USART1 uses PA9/PA10 at 115200 baud, 8 data bits, no parity, one stop bit and no flow control.

3.6. Reading and setting DS1307 time

We replaced the single test routine with `SetTime()` and `GetTime()` in `ds1307.c`. The date and time are passed using this structure:

```
typedef struct {
    uint8_t sec, min, hour, weekday, day, month, year;
} Time;

HAL_StatusTypeDef SetTime(const Time *time);
HAL_StatusTypeDef GetTime(Time *time);
```

The year is stored as 00–99 for 2000–2099. Weekday values are 1–7, with Sunday as 1 in our program. We check the time and date before writing, including the number of days in each month and leap years.

The DS1307 stores its registers in BCD. For example, decimal 59 must be written as `0x59`, not as the ordinary binary value of 59. We added these conversions:

```
static uint8_t bcd(uint8_t v) {
    return (uint8_t)(((v / 10) << 4) | (v % 10));
}
static uint8_t decimal(uint8_t v) {
    return (uint8_t)((v >> 4) * 10 + (v & 15));
}
```

`SetTime()` converts the fields and writes seven bytes starting at register 0. Their order is seconds, minutes, hours, weekday, date, month and year. We clear the clock-halt bit in the seconds register and write the hours in 24-hour format.

```
HAL_I2C_Mem_Write(&hi2c3, 0x68 << 1, 0,
    I2C_MEMADD_SIZE_8BIT, bytes, 7, 50);
```

For the HAL memory read/write functions, we pass `0x68 << 1` for both directions. HAL handles the read/write bit.

`GetTime()` reads the seven registers together and converts them back to decimal. It checks the I2C result, BCD values, calendar values and clock-halt bit. It can also decode 12-hour time if that mode was already set on the chip. An invalid reading returns an error instead of being displayed as a valid time.

We set the time once using `TIME SET YYYY-MM-DD HH:MM:SS`. The normal startup and main loop do not set it again. The program reads and prints the time about once per second.

The terminal showed the seconds advancing. We then reset the STM32 and the time continued without another set command. In the saved test it advanced from 13:46:26 before reset to 13:46:27 afterward. This was a reset test with the RTC still powered, not a test of battery backup after removing USB.

3.7. Showing time on the OLED

We added the supplied `SH1106` and `fonts` files to the project. OLED initialization runs after I2C3 initialization. The display responded at address `0x3C`.

In `ShowClock()`, we call `GetTime()`, clear the old display buffer and format the time and date. The display stage used the following code for the time:

```
snprintf(text, sizeof text, "%02u:%02u:%02u",
         t.hour, t.min, t.sec);
SH1106_GotoXY(20, 18);
SH1106_Puts(text, &Font_11x18, SH1106_COLOR_WHITE);
SH1106_UpdateScreen();
```

The drawing functions change a buffer in STM32 RAM. `SH1106_UpdateScreen()` sends that buffer to the OLED, so calling only `SH1106_Puts()` is not enough to update the physical screen. We clear the buffer before drawing to avoid leaving old characters on it. The screen is refreshed about once per second.

We changed the supplied OLED transfer timeout from 10 ms to 50 ms. A page transfer contains 129 bytes and takes more than 11 ms at 100 kHz, so 10 ms was too short. The long device-readiness timeout was also reduced.

The OLED showed the date and advancing time during the experiment. In the final 3.8 version of this same project, we use the smaller font to leave room for the RFID code as well. The RTC functions and continuous date/time display remain in the project.

3.8. Showing the RFID code

We used the supplied MFRC522 library with SPI4. Our application initializes the reader and checks its VersionReg before polling cards. The module returned `0xB2`. We also checked that it could actually read a card; a register response alone would not prove that.

The added polling code requests a card, runs anticollision to get its code, and then sends HALT:

```
uint8_t code[MFRC522_MAX_LEN] = {0};
TM_MFRC522_Status_t status =
    TM_MFRC522_Request(PICC_REQALL, code);
if (status == MI_OK)
    status = TM_MFRC522_Anticoll(code);
TM_MFRC522_Halt();
```

The receive buffer has 16 bytes because that is the library's maximum transfer length. We do not use the five-byte buffer from the example, which is smaller than the driver's possible transfer.

`PICC_REQALL` wakes a halted card so it can be detected again while it is still held near the reader. We display the five returned bytes using two hexadecimal digits per byte and `Font_7x10`.

Our card gave **B3 C0 8F 34 C8**. The first four bytes are the UID in the supplied library's single-level anticollision routine. The last byte is the XOR check byte, or BCC: `B3 XOR C0 XOR 8F XOR 34 = C8`. The code does not implement the full cascade procedure for longer UIDs.

We presented the card, checked that all five bytes appeared on the OLED, removed it and presented it again. We used one card; no key fob was used.

3.9. Access control and opening log

We made the separate `Lab2_AccessControl` project from the working peripheral project. The added files are `card_presence.c/.h` and `access_control.c/.h`, together with the application code in `main.c`.

Card presence and LEDs

The program polls the reader about every 100 ms. `CardPresence_Update()` remembers the last code and whether a card is present. A new code or a card arriving after removal produces a new-card event. Reading the same held card again does not produce another event. Three missed reads in a row count as removal, so a single missed read does not immediately turn the LED off.

On a new-card event, we compare the code with the allowed list and read the RTC. The main part of the added authorization function is:

```
if (!Access_IsAllowed(access, code))
    return ACCESS_REJECTED;
if (!time)
    return ACCESS_CLOCK_ERROR;
Access_AppendLog(access, time, code);
return ACCESS_GRANTED;
```

We turn on the simulated door only for `ACCESS_GRANTED`. This also makes sure that an opening has a valid timestamp. If the RTC or reader fails, the program keeps or returns the door to the locked state.

Situation	LED3, green PG13	LED4, red PG14	OLED
No card	OFF	OFF	Ready
Card not allowed	ON	OFF	Rejected
Card allowed and time valid	ON	ON	Welcome
Card removed	OFF	OFF	Ready

Deleting the permission of a card that is still present locks the door immediately. Adding a permission requires removing and presenting the card again before it can open the door.

Allowed list and logs

The list holds up to ten five-byte codes and starts empty. We check for duplicates before adding a code. On deletion, the following entries move down to fill the gap. UART input must contain exactly ten hexadecimal digits and a correct BCC.

Each log entry contains the opening date/time, card code and a sequence number. We store at most 100 entries in RAM. `log_head` points to the oldest record. Before the array is full, the next record goes to `(log_head + log_count) % 100`. When it is full, a new record replaces the oldest and advances `log_head`. Reading the records uses the same index calculation to print them oldest first.

Only successful openings are logged. Rejected cards do not add an opening record, and holding a card does not add a record every 100 ms. Both the allowed list and logs are lost after MCU reset. We did not use the AT24C32 for logs because the exercise asks for RAM storage.

UART commands

The commands below are sent from the PC at 115200 baud, 8N1. Each command ends with Enter; the program accepts CR, LF or CRLF. Commands are case-sensitive. We used a desktop serial console to send the text commands and read replies. The same commands can be sent using Hercules.

Command	What it does
HELP	Show the commands
STATUS	Show reader/RTC status, LEDs' door state and counts
SCAN	Show responding I2C addresses
RFID INIT	Initialize the reader again
TIME GET	Print the current RTC time
TIME SET YYYY-MM-DD HH:MM:SS	Set the RTC explicitly
CARD LAST	Print the last read card code
ALLOW ADD B3C08F34C8	Add our card to the allowed list
ALLOW DEL B3C08F34C8	Remove our card from the list
ALLOW LIST	Print allowed codes
LOG LIST	Print the log count and timestamped records
LOG CLEAR	Clear logs and reset their sequence numbers

UART reception uses an interrupt and a 256-byte receive buffer. The main loop assembles a line in a 96-byte buffer and handles the command. Overlong lines are discarded with an error, rather than executing part of a command. OLED updates and log printing are done outside the interrupt.

For `LOG LIST`, we copy the current records to a snapshot and print them a record at a time from the main loop. This allows card polling to continue during a longer log output. The output starts with `LOG BEGIN count=...` and ends with `LOG END`.

Result

We first presented the card with an empty allowed list. The OLED showed Rejected. Green LED3 stayed on while the card was present, and red LED4 stayed off. Both were off after removal.

We then sent `ALLOW ADD B3C08F34C8` and presented the same card again. The OLED showed Welcome and both LEDs turned on. Holding the card did not need another command. Removing it turned both LEDs off. We read the opening records through UART and then removed the permission to show rejection again.

The saved test output includes these real records:

```
EVENT CARD B3C08F34C8 WELCOME
LOG 1 2026-10-30 14:58:02 B3C08F34C8
EVENT CARD REMOVED; DOOR LOCKED
EVENT CARD B3C08F34C8 WELCOME
LOG 2 2026-10-30 14:58:07 B3C08F34C8
```

We also checked duplicate enrollment, an incorrect BCC, deletion of a missing code and a command that exceeded the line buffer. These returned an error or the expected duplicate response without changing the list incorrectly.

Checks and limitations

The experimental Debug builds of both projects finished with zero errors and zero warnings. STM32CubeProgrammer reported that the downloads were verified. We checked actual I2C responses, RTC output, OLED updates, card reads and the LED behavior on the board.

The included C tests check BCD and calendar handling, I2C error returns, held-card/removal state, allowed-list operations and the 100-record log. After 101 test appends, the log contains entries 2–101 in order. This overflow check was done in a host test, not by filming 101 manual card presentations. The tests supplement the hardware experiment.

Battery backup after total power loss was not tested because the coin cell was removed. The application uses a fixed list of ten allowed codes, a 100-record RAM log and the supplied library's four-byte UID path. It simulates a door with LEDs; it does not drive a physical lock.

Videos

1. **3.6 — DS1307 time on terminal:** Google Drive video
2. **3.7 — Date and time on OLED:** Google Drive video
3. **3.8 — Five-byte RFID code on OLED:** Google Drive video
4. **3.9 — RFID access control and logs:** Google Drive video

References

- Instructor's `Lab2_IT4210E.pdf`, `Thuc hanh 2.pdf` and current assignment announcement.
- Supplied DS1307 datasheet, especially the clock/calendar registers and I2C address sections.
- Supplied AT24C32 datasheet, for capacity, address inputs and I2C operation.
- Supplied SH1106 and fonts libraries: Tilen Majerle, Alexander Lutsai and the SH1106 adaptation credited to ControllersTech. Original notices are kept in the source.
- Supplied MFRC522 library: Tilen Majerle, with the STM32F429/HAL adaptation by TrungNL. We retained the earlier integration's HAL enum-type correction and the library notices.
- STMicroelectronics HAL, CMSIS, startup and linker support supplied with the projects. Our application additions are explained in sections 3.6–3.9 above.